

UNIVERSIDAD DE GUADALAJARA

Centro Universitario de Tonalá

Ingeniería en Ciencias Computacionales

7^{to} semestre

Tema: Proyecto Final (EcoCompress S3)



PRESENTA:

Gerardo Josue Rubio Calderon

Matricula:

220321903

Docente:

VICTOR HUGO VEGA FREGOSO

Materia:

Computación Sustentable

1. Introducción y Propósito del Proyecto

Introducción:

En la era actual del Big Data, el almacenamiento en la nube ha crecido de forma exponencial. Sin embargo, una gran parte de estos datos se almacena en formatos eficientes o sin compresión, lo que deriva en un desperdicio masivo de recursos digitales. Desde la perspectiva de la Computación Sustentable, este almacenamiento innecesario se traduce en una mayor cantidad de servidores activos, mayor costo de suscripción, mayor consumo eléctrico en los centros de datos y, por consecuencia, una huella de carbono digital mas elevada.

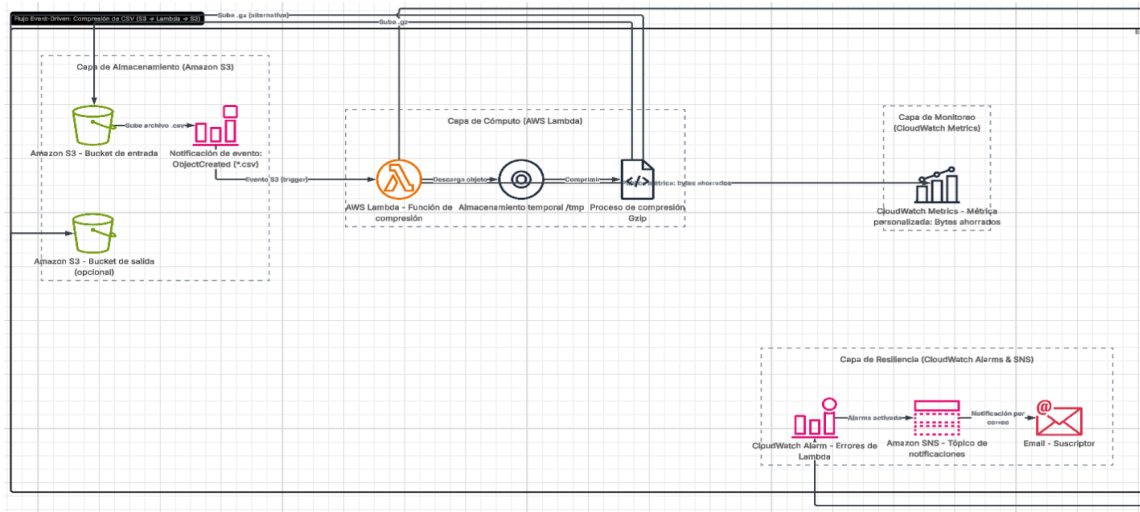
Propósito:

EcoCompress S3 nace como una solución automatizada y serverless diseñada para optimizar el uso de recursos en Amazon Web Services (AWS). El objetivo principal es interceptar cada archivo subido a un bucket de S3 y comprimirlo automáticamente en tiempo real.

Este proyecto backend busca demostrar que es posible reducir los costos operativos y el impacto ambiental mediante la automatización de la infraestructura (IaC) y el uso de servicios bajo demanda que solo consumen energía durante el procesamiento activo.

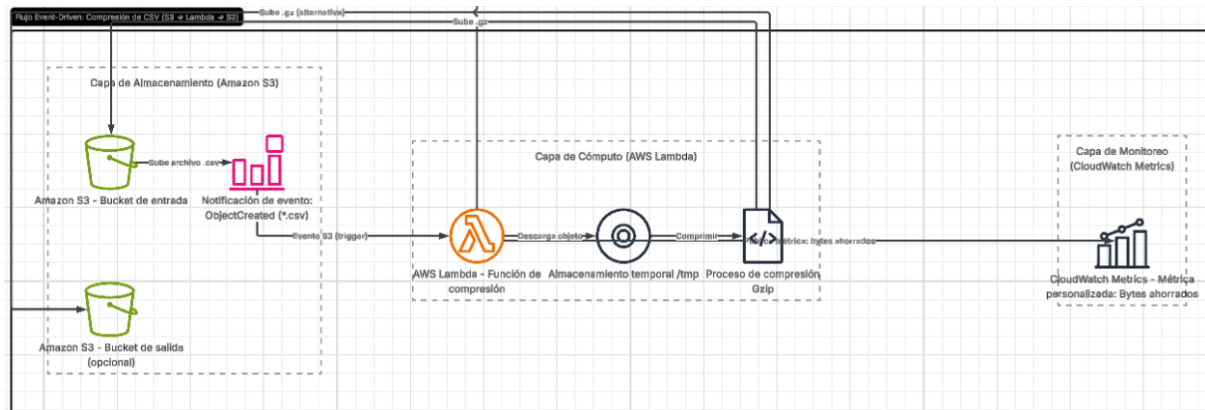
2. Diagrama de Arquitectura

La arquitectura es de tipo Event-Driven (dirigida por eventos), lo que garantiza que ningún recurso sea utilizado si no hay trabajo que realizar.



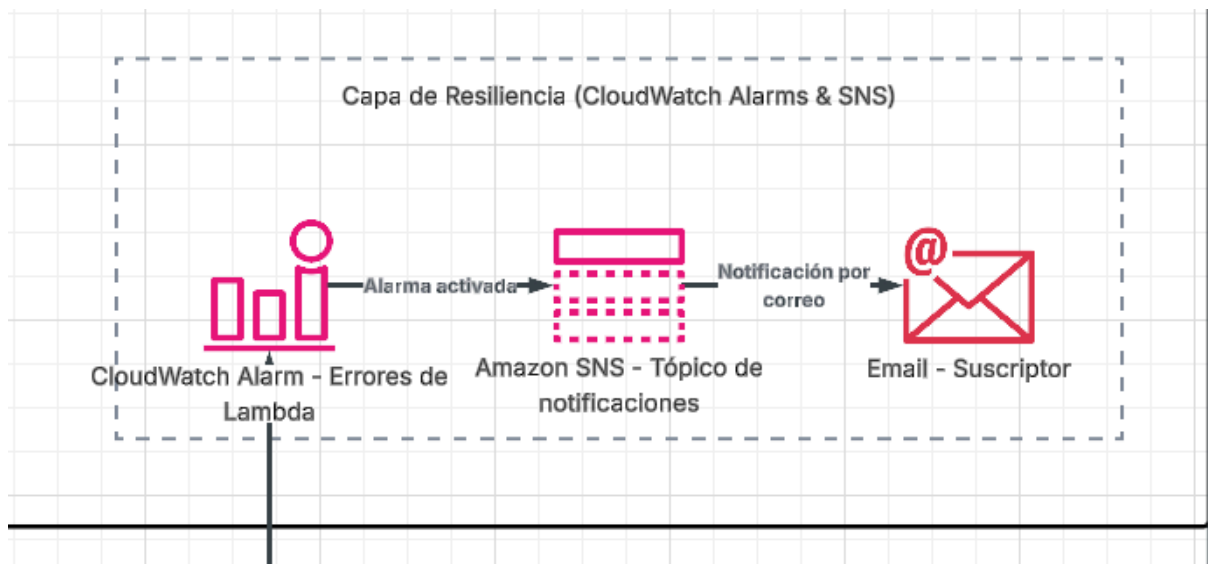
1. El punto de entrada sería el S3. En donde al recibir un archivo, se genera una notificación inmediata.
2. La capa de cómputo (AWS Lambda): se activa al recibir el evento de S3. Por lo que descarga el archivo a un entorno temporal, lo comprime usando el algoritmo Gzip y lo sube de nuevo al bucket con la extensión .gz.
3. Seguido de la capa de cómputo, la Lambda envía datos personalizados a Cloudwatch Metrics sobre la cantidad de bytes ahorrados después de cada operación.

Arquitectura de cómo se maneja el script.



Arquitectura de la capa de resiliencia

Para esta capa, se utiliza CloudWatch Alarms & SNS: Mediante logs, existe un sistema de vigilancia que monitorea si la Lambda genera errores. En caso de falla, CloudWatch activa una alarma que envía una notificación instantánea vía correo electrónico a través de Amazon SNS.



3. Justificación de Decisiones Técnicas

Para el desarrollo de EcoCompress S3, se tomaron decisiones basadas en tres pilares: Automatización, Sustentabilidad y Escalabilidad. Seguido de esto, detallare los motivos por los cuales seleccione cada servicio:

¿Por qué elegí AWS Lambda en vez de Amazon EC2? Opte por una arquitectura Serverless con AWS Lambda en lugar de servidores virtuales, porque una instancia de EC2 requiere estar encendida 24/7 para procesar archivos en cualquier momento, lo que consume energía en todo momento independientemente de la carga de trabajo. En cambio, usando Lambda, utilizó un modelo de “Pago por Uso” y “Cómputo bajo demanda”. Y esto impacta de buena manera porque el servidor solo consume recursos de CPU y memoria durante los segundos que pueda llegar a durar la compresión. Por lo tanto, si no hay archivos, el consumo energético es cero.

¿Por qué elegí Python como lenguaje de automatización? Durante todo el semestre hicimos uso de Python y la librería Boto3 para la creación de scripts para nuestros servicios. Así como también Python es un lenguaje de alto nivel con una eficiencia notable en tareas de procesamiento de texto y archivos. Y en cuento Boto3, esta librería nos permite una comunicación directa y ligera con las APIs de AWS.'

Gzip como algoritmo de compresión: Utilice la librería gzip de python, y lo elegí por ser un estándar de código abierto, ampliamente compatible y con un excelente equilibrio entre velocidad de procesamiento y tasa de reducción de tamaño. Por lo cual permite reducir el tiempo de ejecución de Lambda, reduciendo el costo y el consumo de cómputo.

Monitoreo Reactivo con CloudWatch & SNS: En lugar de realizar revisiones manuales, el sistema es capaz de "notificar" su estado. Lo que garantiza la resiliencia del proyecto, permitiendo que el administrador actúe sólo cuando la alarma se activa, optimizando también el recurso humano.

Análisis de Ventajas y Desventajas

Servicio	Ventajas	Desventajas
AWS Lambda	Escalabilidad automática, cero mantenimiento de servidores, costo nulo en reposo.	Límite de 15 minutos por ejecución (insuficiente para archivos de varios Gigabytes).
Amazon S3	Durabilidad del 99.9% y costo de almacenamiento extremadamente bajo.	Latencia mínima en el disparo del evento (trigger).
Amazon SNS	Notificaciones instantáneas y multicanal.	Requiere confirmación manual por parte del usuario para evitar SPAM.

4. Análisis de Costos (Estimado Anual)

El modelo de costos de EcoCompress S3 se basa en la filosofía Serverless, donde solo se paga por lo que se consume. Para este análisis, se considera un escenario de uso moderado para una pequeña o mediana empresa (PyME) que procesa aproximadamente 10,000 archivos al mes (aprox. 50 GB de datos originales).

Servicio AWS	Concepto	Costo Mensual (Free Tier)	Costo Mensual (Uso Real)
Amazon S3	Almacenamiento (5 GB iniciales) y Solicitudes PUT/GET.	\$0.00 USD	\$0.15 USD
AWS Lambda	10k ejecuciones (aprox. 1 sec c/u) y 128MB RAM.	\$0.00 USD	\$0.02 USD
CloudWatch	1 Alarma activa y métricas personalizadas.	\$0.00 USD	\$0.10 USD
Amazon SNS	Envío de notificaciones vía E-mail.	\$0.00 USD	\$0.00 USD
TOTAL		\$0.00 USD	\$0.27 USD

Proyección Anual y Ahorro Sustentable

Costo Total Estimado (Fuera de Capa Gratuita): Aproximadamente \$3.24 USD al año.

Análisis de Inversión: El costo operativo es insignificante comparado con el beneficio. Si la aplicación logra reducir el almacenamiento total de 500 GB a 100 GB en un año, el ahorro en costos de almacenamiento de S3 compensa con creces el gasto de ejecución de la Lambda.

Por lo cual podemos ver que este proyecto podría ser altamente rentable. Desde la perspectiva de la computación sustentable, no solo ahorramos dinero, sino que optimizamos el uso de hardware físico en la nube, reduciendo la necesidad de mantener discos duros activos para datos que pueden ser almacenados de forma eficiente en formatos comprimidos.

Pruebas de Resiliencia (Incluyendo evidencia)

Escenario 1: Prevención de Bucles.

Un error común en arquitecturas dirigidas por eventos es que la Lambda, al subir el archivo comprimido, dispare un nuevo evento de "Objeto Creado", entrando en un bucle infinito de ejecuciones y costos.

La lambda tardo 1.80 ms en decidir que no iba a trabajar de más. Esto confirma que la lógica de validación de extensiones funciona antes de realizar cualquier descarga de datos, minimizando el uso de recursos de red y CPU.

En esta prueba, para demostrar la Resiliencia, adjunte un archivo ya comprimido, porque el archivo acaba en .gz, asi que subiremos el archivo, y veremos si se hace bucle o no.

```
PS C:\Users\gerar\desktop\computacion-sustentable> aws s3 cp "C:\Users\gerar\Downloads\wordpress-6.9.4.tar.gz" s3://ecocompress-s3-storage-gerardo/
upload: ...Downloads\wordpress-6.9.4.tar.gz to s3://ecocompress-s3-storage-gerardo/wordpress-6.9.4.tar.gz
```

Ahora ingrese a CloduWatch para verificar si hubo un error, o no.

►	Marca temporal	Mensaje
		No hay eventos antiguos en este momento. Volver a intentar
►	2026-05-10T06:15:32.354Z	INIT_START Runtime Version: python:3.9.v133 Runtime Version ARN: arn:aws:lambda:us-east-1::runtime:b46f7bc0f3da8071d1b824471f2c69c876...
►	2026-05-10T06:15:32.880Z	START RequestId: b07a202f-1c19-498d-b7fe-da19844a0a3d Version: \$LATEST
►	2026-05-10T06:15:32.880Z	[*] El archivo wordpress-6.9.4.tar.gz ya esta comprimido.
►	2026-05-10T06:15:32.882Z	END RequestId: b07a202f-1c19-498d-b7fe-da19844a0a3d
►	2026-05-10T06:15:32.882Z	REPORT RequestId: b07a202f-1c19-498d-b7fe-da19844a0a3d Duration: 1.80 ms Billed Duration: 524 ms Memory Size: 128 MB Max Memory Used:...
		No hay eventos recientes en este momento. <i>Reintento automático en pausa.</i> Reanudar

Como nos muestran los logs, podemos ver que la lambda tardo 1.80 ms en decidir que no iba a trabajar de más. Esto confirma que la lógica de validación de extensiones funciona antes de realizar cualquier descarga de datos, minimizando el uso de recursos de red y CPU.

Escenario 2: Notificación de Fallos Criticos (Alarma SNS)

En este escenario lo que realizare sera modificar en el servicio de Lambda mi funcion lambda_function.py, le agregare algo boto3 para que el sistema no comprenda a que libreria estoy llamando. Al momento de marcar como deploy, se esperaria que CloudWatch mande una alerta de error. Y por consiguiente el sistema SNS nos notifique por correo que hay un error en nuestra sistema.

```

import boto3_hola
import gzip
import os
import shutil

s3_client = boto3.client('s3')
cloudwatch = boto3.client('cloudwatch')

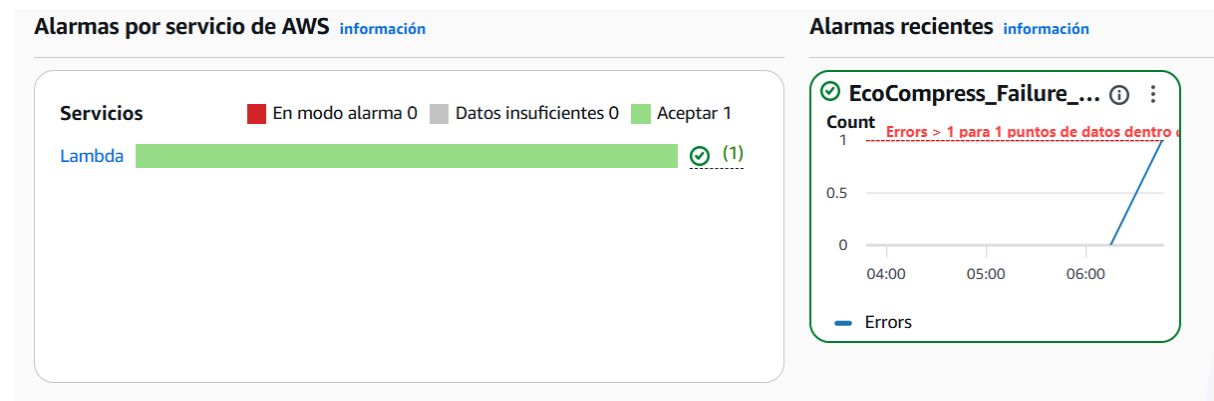
def handler(event, context):
    #Aqui guardamos detallitos de los archivos
    bucket_name = event['Records'][0]['s3']['b
    file_key = event['Records'][0]['s3']['obje

    #Nadita de nada: en caso de que el archivo
    if file_key.endswith('.gz'):
        print(f"[*] El archivo {file_key} ya e
        return {'status': 'skipped', 'reason':

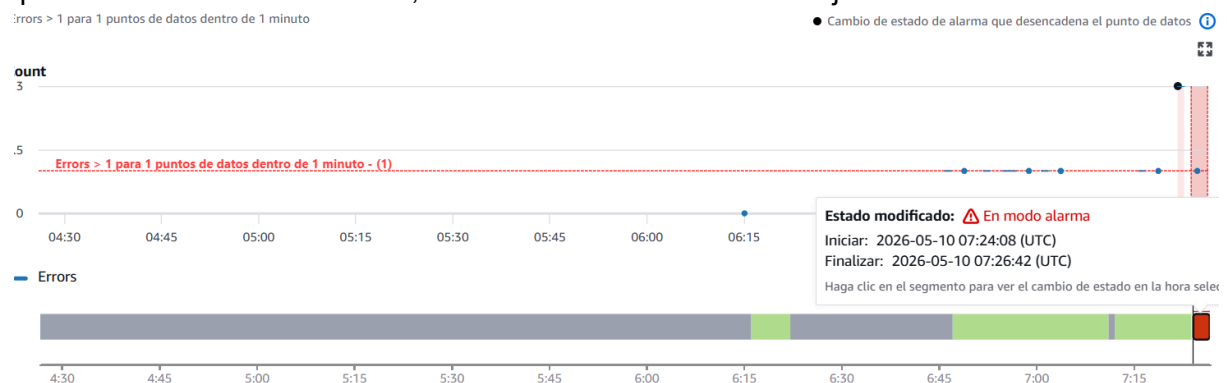
```

Despues de darle a deploy, en Cloudwatch nos aparece esto:

Hola



En las alarmas de CloudWatch, aparecera en la grafica puntitos, que reflejaran los errores que se han estado cometiendo, asi como tambien se volvera rojo.



Paneles

Alarmas 🚨 1 ✅ 0 ⋮ 0

▶ Operaciones de inteligencia artificial

▶ Observabilidad de

☐	Nombre	Estado
☐	EcoCompress_Failure_Alarm	🚨 En modo alarma

Por ultimo, verificamos nuestro correo y efectivamente, aparecera el correo donde nos indique el error:

ALARM: "EcoCompress_Failure_Alarm" in US East (N. Virginia) External Recibidos x

AWS Notifications

para mí

1:24 (hace 3 minutos) ☆ 😊 ↶ ⋮

Parece que este mensaje está en inglés

×

Traducir al español

You are receiving this email because your Amazon CloudWatch Alarm "EcoCompress_Failure_Alarm" in the US East (N. Virginia) region has entered the ALARM state, because "Threshold Crossed: 1 datapoint [3.0 (10/05/26 07:22:00)] was greater than the threshold (1.0)." at "Sunday 10 May, 2026 07:24:08 UTC".

View this alarm in the AWS Management Console:
https://us-east-1.console.aws.amazon.com/cloudwatch/deeplink.js?region=us-east-1#alarmsV2:alarm/EcoCompress_Failure_Alarm

Alarm Details:

- Name: EcoCompress_Failure_Alarm
- Description: Alarma si la Lambda EcoCompress falla.
- State Change: OK -> ALARM
- Reason for State Change: Threshold Crossed: 1 datapoint [3.0 (10/05/26 07:22:00)] was greater than the threshold (1.0).
- Timestamp: Sunday 10 May, 2026 07:24:08 UTC
- AWS Account: 137303519813
- Alarm Arn: arn:aws:cloudwatch:us-east-1:137303519813:alarm:EcoCompress_Failure_Alarm

Threshold:

- The alarm is in the ALARM state when the metric is GreaterThanThreshold 1.0 for at least 1 of the last 1 period(s) of 60 seconds.

Escenario 3: Se comprimen los archivos diminutos?

Se implementó una lógica de control basada en el tamaño del objeto (payload) para evitar el desperdicio de ciclos de reloj de la CPU en archivos cuya tasa de compresión no justifica el gasto energético del procesamiento. Por lo cual se estableció un umbral (threshold) de 10 KB. Si el archivo es inferior a este peso, el sistema aborta la operación de compresión. Esto se fundamenta en que el *overhead* de inicialización del entorno de ejecución de la Lambda y la escritura en disco de un archivo minúsculo consume más energía de la que se ahorra en el almacenamiento final.

holachavales

10/05/2026 01:33 a. m.

Text Document

1 KB

```
PS C:\Users\gerar\desktop\computacion-sustentable> aws s3 cp "C:\Users\gerar\Downloads\holachavales.txt" s3://ecocompress-s3-storage-gerardo/
upload: ..\..\Downloads\holachavales.txt to s3://ecocompress-s3-storage-gerardo/holachavales.txt
```

▶ 2026-05-10T07:35:28.320Z

INIT_START Runtime Version: python:3.9.v133 Runtime Version ARN: arn:aws:lambda:us-east-1::runtime:b46f7bc0f3da8071d1b824471f2c69c876...

▶ 2026-05-10T07:35:28.841Z

START RequestId: 59d0aa71-6e8c-48d5-a595-c707427bb149 Version: \$LATEST

▶ 2026-05-10T07:35:29.124Z

[*] Archivo holachavales.txt muy chico. Omitiremos este archivo para ahorar CPU.

▼ 2026-05-10T07:35:29.126Z

END RequestId: 59d0aa71-6e8c-48d5-a595-c707427bb149

END RequestId: 59d0aa71-6e8c-48d5-a595-c707427bb149

▶ 2026-05-10T07:35:29.126Z

REPORT RequestId: 59d0aa71-6e8c-48d5-a595-c707427bb149 Duration: 284.22 ms Billed Duration: 803 ms Memory Size: 128 MB Max Memory Use...

No hay eventos recientes en este momento. *Reintentó automáticamente en pausa.* [Reanudar](#)

En el log podemos verificar que el archivo que subi es muy chico, por lo cual, el script decidió omitir el archivo para ahorrar CPU.

Conclusión Final

El desarrollo de EcoCompress S3 ha permitido integrar con éxito los pilares fundamentales de la ingeniería de software moderna: la automatización, la eficiencia operativa y el compromiso ambiental. A través de la implementación de esta arquitectura *serverless*, se han validado las siguientes conclusiones:

- **Eficiencia Energética y Sustentabilidad:** Al migrar de un modelo de computación tradicional (servidores encendidos 24/7) a un modelo dirigido por eventos (AWS Lambda), se optimiza el consumo de recursos de los centros de datos. El sistema solo consume energía en el momento preciso del procesamiento, reduciendo directamente la huella de carbono digital asociada al almacenamiento de datos ineficientes.
- **Automatización y Resiliencia:** La creación de la infraestructura mediante scripts (IaC) no solo cumple con la Regla 1 del proyecto, sino que elimina el error humano y permite despliegues rápidos y consistentes. La integración de CloudWatch y SNS garantiza que la arquitectura no sea solo una herramienta pasiva, sino un sistema inteligente capaz de monitorear su propio desempeño y alertar sobre anomalías.
- **Viabilidad Económica:** El análisis de costos demuestra que la optimización no tiene por qué ser costosa. Con un gasto operativo proyectado de menos de \$4.00 USD anuales (fuera de capa gratuita), la solución se paga sola al reducir los costos de almacenamiento a largo plazo en Amazon S3.

En última instancia, este proyecto demuestra que la Computación Sustentable no es solo una teoría académica, sino una práctica técnica viable que permite construir sistemas más robustos, económicos y responsables con el medio ambiente.

Link de mi repositorio de GitHub: <https://github.com/snipergerard/EcoCompress-S3>